



7.1.1 Where do sparse linear systems come from?

7.1.1 Poisson's equation, Part 1

fit width



Many computational engineering and science applications start with some law of physics that applies to some physical problem. This is mathematically expressed as a Partial Differential Equation (PDE). We here will use one of the simplest of PDEs, Poisson's equation on the domain Ω in two dimensions:

$$-\Delta u = f.$$

In two dimensions this is alternatively expressed as

$$-\frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2} = f(x, y) \quad (7.1.1)$$

with Dirichlet boundary condition $\partial\Omega = 0$ (meaning that $u(x, y) = 0$ on the boundary of domain Ω). For example, the domain may be the square $0 \leq x, y \leq 1$, $\partial\Omega$ its boundary, and the question may be a membrane with f being some load from, for example, a sound wave.

Since this course does not require a background in the mathematics of PDEs, let's explain the gist of all this in layman's terms.

- We want to find the function u that satisfies the conditions specified by (7.1.1). It is assumed that u is appropriately differentiable.
- For simplicity, let's assume the domain is the square with $0 \leq x \leq 1$ and $0 \leq y \leq 1$ so that the boundary Ω is the boundary of this square. We assume that on the boundary the function equals zero.
- It is usually difficult to analytically determine the continuous function u that solves such a "boundary value problem" (except for very simple examples).
- To solve the problem computationally, the problem is "discretized". What this means for our example is that a mesh is laid over the domain, values for the function u at the mesh points are approximated, and the operator is approximated. In other words, the continuous domain is viewed as a mesh instead, as illustrated in Figure 7.1.1.1 (Left). We will assume an $N \times N$ mesh of equally spaced points, where the distance between two adjacent points is $h = 1/(N + 1)$. This means the mesh consists of points $\{(x_i, \psi_j)\}$ with $x_i = (i + 1)h$ for $i = 0, 1, \dots, N - 1$ and $\psi_j = (j + 1)h$ for $j = 0, 1, \dots, N - 1$.

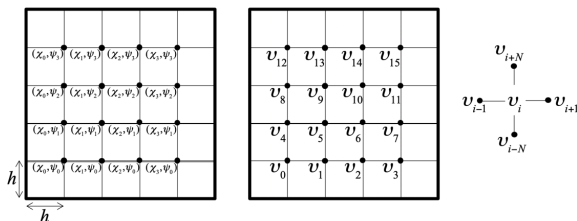


Figure 7.1.1.1. 2D mesh.

- If you do the math, details of which can be found in Subsection 7.4.1, you find that the problem in (7.1.1) can be approximated with a linear equation at each mesh point:

$$\frac{-u(x_i, \psi_{j-1}) - u(x_{i-1}, \psi_j) + 4u(x_i, \psi_j) - u(x_{i+1}, \psi_j) - u(x_i, \psi_{j+1}))}{h^2} = f(x_i, \psi_i).$$

The values in this equation come from the "five point stencil" illustrated in Figure 7.1.1.1 (Right).

fit width



- $$-v_{i-N} - v_{i-1} + 4v_i - v_{i+1} - v_{i+N} = h^2 \phi_i$$

$$v_i = \frac{h^2 \phi_i + v_{i-N} + v_{i-1} + v_{i+1} + v_{i+N}}{4}$$

fit width


$$\begin{array}{ccccccccc}
4v_0 & -v_1 & & & -v_4 & & & & = h^2 \phi_0 \\
-v_0 & +4v_1 & -v_2 & & -v_5 & & & & = h^2 \phi_1 \\
& -v_1 & +4v_2 & -v_3 & & -v_6 & & & = h^2 \phi_2 \\
& & -v_2 & +4v_3 & & & -v_7 & & = h^2 \phi_3 \\
-v_0 & & & & +4v_4 & -v_5 & & -v_8 & = h^2 \phi_4 \\
& \ddots & & & \ddots & \ddots & \ddots & \ddots & \vdots
\end{array}$$
$$\left(\begin{array}{ccc|ccc|ccc} 4 & -1 & & & & & & & \\ -1 & 4 & -1 & & & & & & \\ & -1 & 4 & -1 & & & & & \\ & & -1 & 4 & & & & & \\ \hline -1 & & & & 4 & -1 & & & \\ & & & & -1 & 4 & -1 & & \\ & & & & & -1 & 4 & & \\ \hline & & -1 & & & & & 4 & \ddots \\ & & & & & & & & \ddots \\ \hline & & & & -1 & & & & \\ & & & & & & & 4 & \ddots \\ & & & & & & & & \ddots \end{array} \right) \left(\begin{array}{c} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \\ v_7 \\ v_8 \\ \vdots \end{array} \right) = \left(\begin{array}{c} h^2 \phi_0 \\ h^2 \phi_1 \\ h^2 \phi_2 \\ h^2 \phi_3 \\ h^2 \phi_4 \\ h^2 \phi_5 \\ h^2 \phi_6 \\ h^2 \phi_7 \\ h^2 \phi_8 \\ \vdots \end{array} \right).$$

- Discretize the domain $0 \leq \chi, \psi \leq 1$ by creating an $(N+2) \times (N+2)$ mesh of points.
- An $(N+2) \times (N+2)$ array U holds the values $u(\chi_i, \psi_i)$ *plus the boundary around it*.
- Create an $(N+2) \times (N+2)$ array F that holds the values $f(\chi_i, \psi_j)$ (plus, for convenience, extra values that correspond to the boundary).
- Set all values in U to zero. This initializes the last rows and columns to zero, which captures the boundary condition, and initializes the rest of the values at the mesh points to zero.
- Repeatedly update all interior points with the formula

$$U(i, j) = \left(h^2 \underbrace{F(i, j)}_{f(\chi_i, \psi_j)} + \underbrace{U(i, j-1)}_{u(\chi_i, \psi_{j-1})} + \underbrace{U(i-1, j)}_{u(\chi_{i-1}, \psi_j)} + \underbrace{U(i+1, j)}_{u(\chi_{i+1}, \psi_j)} + \underbrace{U(i, j+1)}_{u(\chi_i, \psi_{j+1})} \right) / 4$$

until the values converge.

- Bingo! You have written your first iterative solver for a sparse linear system.
- Test your solver with the problem where $f(\chi, \psi) = (\alpha^2 + \beta^2)\pi^2 \sin(\alpha\pi\chi) \sin(\beta\pi\psi)$.
- Hint: if x and y are arrays with the vectors x and y (with entries χ_i and ψ_j), then `mesh(x, y, U)` plots the values in U .

▼ [Hint](#) ▼ [Solution](#)

[Assignments/Week07/answers/Poisson_Jacobi_iteration.m](#).

When you execute the script, in the COMMAND WINDOW enter "RETURN" to advance to the next iteration.

An outline for a matlab script can be found in

[Assignments/Week07/matlab/Poisson_Jacobi_iteration.m](#).

When you execute the script, in the COMMAND WINDOW enter "RETURN" to advance to the next iteration.

Remark 7.1.1.2. In [Homework 7.2.1.4](#) we store the vectors u and f as they appear in [Figure 7.1.1.1](#) as 2D arrays. This captures the fact that a 2d array of numbers isn't necessarily a matrix. In this case, it is a vector that is stored as a 2D array because it better captures how the values to be computed relate to the physical problem from which they arise.

7.1.1 What is a sparse matrix

fit width



Remark 7.1.1.3. The point of this launch is that many problems that arise in computational science require the solution to a system of linear equations $Ax = b$ where A is a (very) sparse matrix. Often, the matrix does not even need to be explicitly formed and stored.

Remark 7.1.1.4. Wilkinson defined a sparse matrix as any matrix with enough zeros that it pays to take advantage of them.

The problem used to motivate in this unit were suggested to us by Isaac Lee. You may enjoy other materials of his by visiting

<https://crunchingnumbers.live/2017/07/09/iterative-methods-part-2/>.